

Functions, loops and dictionaries

Machine Learning in Finance · Session 2

One return is easy

Last session you stored prices in a **list**, indexed it, and computed a **simple return** from two prices. That return is a single line:

Python Code

```
prices = [183.56, 182.19, 179.87, 179.15, 183.48]  
(prices[1] - prices[0]) / prices[0]
```

```
-0.007463499673131426
```

That is the return for **one** day, from the first close to the second.

A year is 252 numbers

To study a stock you want **every** day's return. Apple traded about **252 days** in 2024, so that is 251 returns.

Writing them out one line at a time, $(\text{prices}[1] - \text{prices}[0]) / \text{prices}[0]$, then $(\text{prices}[2] - \text{prices}[1]) / \text{prices}[1]$, and so on, is not realistic.

You need a way to say: do this once for every day. That is a **loop**, and it is where we start today.

Today

By the end of this session you can:

- repeat work automatically with a **for loop** and a **while loop**
- make choices with **if**
- package a calculation into a **function** you write yourself
- organise results by name in a **dictionary**
- combine all of it to measure risk properly, as **volatility**

And you continue the course **case**: turning last session's rough risk gauge into a real one.

1 • For loops

PART 1

Doing something once for every item in a list.

How a loop runs

Step through it one click at a time. Watch the highlighted line, and how the variable and the output change:

```
prices = [183.56, 182.19, 179.87]
for price in prices:
    print(price)
```

How a loop runs

Step through it one click at a time. Watch the highlighted line, and how the variable and the output change:

```
prices = [183.56, 182.19, 179.87]
for price in prices:
    print(price)
```

VARIABLES

price = 183.56

OUTPUT

take the first item

How a loop runs

Step through it one click at a time. Watch the highlighted line, and how the variable and the output change:

```
prices = [183.56, 182.19, 179.87]
for price in prices:
    print(price)
```

VARIABLES

price = 183.56

OUTPUT

183.56

run the body

How a loop runs

Step through it one click at a time. Watch the highlighted line, and how the variable and the output change:

```
prices = [183.56, 182.19, 179.87]
for price in prices:
    print(price)
```

VARIABLES

price = 182.19

OUTPUT

183.56

take the next item

How a loop runs

Step through it one click at a time. Watch the highlighted line, and how the variable and the output change:

```
prices = [183.56, 182.19, 179.87]
for price in prices:
    print(price)
```

VARIABLES

price = 182.19

OUTPUT

183.56

182.19

run the body

How a loop runs

Step through it one click at a time. Watch the highlighted line, and how the variable and the output change:

```
prices = [183.56, 182.19, 179.87]
for price in prices:
    print(price)
```

VARIABLES

price = 179.87

OUTPUT

183.56

182.19

take the next item

How a loop runs

Step through it one click at a time. Watch the highlighted line, and how the variable and the output change:

```
prices = [183.56, 182.19, 179.87]
for price in prices:
    print(price)
```

VARIABLES

price = 179.87

OUTPUT

183.56

182.19

179.87

run the body

How a loop runs

Step through it one click at a time. Watch the highlighted line, and how the variable and the output change:

```
prices = [183.56, 182.19, 179.87]
for price in prices:
    print(price)
```

VARIABLES

no items left

OUTPUT

183.56

182.19

179.87

the list is finished, so the loop stops

A first loop

Now run it for real. Instead of printing prices one by one, let Python walk the list:

Python Code

```
prices = [183.56, 182.19, 179.87]
for price in prices:
    print(price)
```

183.56

182.19

179.87

for price in prices: reads as “for each price in the list”. The indented line runs once per item.

The loop variable

Each time round, **price** holds the **next** value in the list:

Python Code

```
prices = [183.56, 182.19, 179.87]
for price in prices:
    print("close:", price)
```

```
close: 183.56
close: 182.19
close: 179.87
```

price is an ordinary variable, and you choose its name.

Indentation marks the body

Python knows what is **inside** the loop from the indentation:

```
for price in prices:  
    print(price)      # indented: inside the loop, runs each time  
print("done")        # not indented: runs once, after the loop
```

The colon `:` opens the loop; the indented lines below it are its body.

Predict the output

PREDICT

Before running, decide how many lines this prints, and what they are.

Python Code

```
prices = [10, 20, 30, 40]
for p in prices:
    print(p * 2)
```

One line for each item, or something else? Press Run to check.

Adding up a list, by hand

A loop can build a running **total**. Start at zero, then add each price:

Python Code

```
prices = [183.56, 182.19, 179.87, 179.15, 183.48]
total = 0
for price in prices:
    total = total + price
total
```

908.25

This is exactly what `sum(prices)` does. Now you can see inside it.

The accumulator pattern

That shape comes up constantly:

- start a variable at a neutral value (0 for a sum)
- update it once per item, inside the loop
- read it after the loop

Set up, then loop and update, then use the result. Summing, counting, and averaging all follow this one pattern.

A shorthand: +=

Updating a variable from itself, `total = total + r`, is so common that Python gives it a shorthand:

Python Code

```
total = 0
for r in [0.01, -0.02, 0.03]:
    total += r      # same as: total = total + r
total
```

```
0.019999999999999997
```

`total += r` means `total = total + r`. The same works for `-=`, `*=`, and `/=`. We will use it for accumulators from here on.

Looping a fixed number of times

`range(n)` gives the whole numbers `0, 1, ..., n - 1` to loop over:

Python Code

```
for i in range(5):  
    print(i)
```

```
0  
1  
2  
3  
4
```

Use it when you care about **positions**, not the values themselves.

range with a start

`range(start, stop)` begins at **start** and stops **before stop**:

Python Code

```
for i in range(1, 5):  
    print(i)
```

```
1  
2  
3  
4
```

Just like slicing, the stop value is not included.

Reaching two days at once

To compare each day with the one before it, loop over positions and use `i` and `i - 1`:

Python Code

```
prices = [183.56, 182.19, 179.87, 179.15, 183.48]
for i in range(1, len(prices)):
    print(prices[i], "after", prices[i - 1])
```

```
182.19 after 183.56
179.87 after 182.19
179.15 after 179.87
183.48 after 179.15
```

Starting from an empty list

To collect results, begin with an **empty list** and grow it with `.append`:

Python Code

```
returns = []          # empty, ready to fill
returns.append(0.012)
returns.append(-0.004)
returns
```

```
[0.012, -0.004]
```

`[]` is a list with nothing in it yet. This is how you gather values as a loop runs.

Every daily return, at once

Put the two ideas together: loop over positions, and append each return:

Python Code

```
prices = [183.56, 182.19, 179.87, 179.15, 183.48]
returns = []
for i in range(1, len(prices)):
    r = (prices[i] - prices[i - 1]) / prices[i - 1]
    returns.append(r)
returns
```

```
[-0.007463499673131426,
 -0.012733959053735075,
 -0.004002890976816584,
 0.024169690203739793]
```

Your turn 1

Use a loop to add up all the returns into `total`. Start `total` at zero, and add each return inside the loop.

Python Code

```
returns = [0.012, -0.004, 0.008, -0.011, 0.006]
```

```
total = ...  
for r in returns:  
    ...  
total
```

Your turn 1: answer

The accumulator pattern, on the returns:

```
returns = [0.012, -0.004, 0.008, -0.011, 0.006]

total = 0
for r in returns:
    total += r
total
```

Result: 0.011. Start at zero, add each return once, read the total after the loop.

2 • Making choices

PART 2

Running code only when a condition is true.

The if statement

Last session a comparison gave **True** or **False**. **if** acts on that answer:

Python Code

```
price = 260
if price > 250:
    print("above 250")
```

above 250

The indented block runs only when the condition is **True**. Change **price** to **240** and run it again.

else: the other case

`else` handles what happens when the condition is `False`:

Python Code

```
price = 240
if price > 250:
    print("above 250")
else:
    print("250 or below")
```

250 or below

elif: more than two cases

elif (short for else-if) adds further cases, checked in order:

Python Code

```
r = -0.02
if r > 0:
    print("up day")
elif r < 0:
    print("down day")
else:
    print("flat day")
```

down day

Predict the branch

PREDICT

What does this print? Decide before you run it.

Python Code

```
r = 0.0
if r > 0:
    print("up day")
elif r < 0:
    print("down day")
else:
    print("flat day")
```

Which branch is true when **r** is exactly zero?

if inside a loop

Put the two together: check a condition for every item. Count the up-days:

Python Code

```
returns = [0.012, -0.004, 0.008, -0.011, 0.006]
up_days = 0
for r in returns:
    if r > 0:
        up_days += 1
up_days
```

3

The **if** sits inside the loop body, indented once more. **up_days** grows only on positive days.

Indentation stacks

Each `:` opens a block, indented one step further than the last:

```
for r in returns:           # loop body: indented once
    if r > 0:                 # if body: indented twice
        up_days += 1
```

Read the indentation to see what belongs to what.

Your turn 2

Count how many returns were **negative** (a down day).

Python Code

```
returns = [0.012, -0.004, 0.008, -0.011, 0.006]
```

```
down_days = 0
for r in returns:
    ...
down_days
```

Your turn 2: answer

The up-day count, with the comparison flipped:

```
returns = [0.012, -0.004, 0.008, -0.011, 0.006]

down_days = 0
for r in returns:
    if r < 0:
        down_days += 1
down_days
```

Result: 2. Two of the five returns are below zero.

3 • While loops

PART 3

Repeating for as long as a condition stays true.

When you do not know how many times

A **for loop** runs once per item. A **while loop** runs for as long as a condition holds, however many times that takes.

How many years to grow 100 into 150 at 10% a year?

Python Code

```
value = 100
years = 0
while value < 150:
    value *= 1.10
    years += 1
years
```

5

You did not have to know the answer in advance. The loop keeps going until **value** reaches 150.

Watching a while loop

Step through it. The condition is checked before each pass; watch `value` and `years` change:

```
value = 100
years = 0
while value < 150:
    value *= 1.10
    years += 1
```

Watching a while loop

Step through it. The condition is checked before each pass; watch **value** and **years** change:

```
value = 100
years = 0
while value < 150:
    value *= 1.10
    years += 1
```

VARIABLES

value = 100.00

years = 0

100 < 150 ? yes, enter the loop

Watching a while loop

Step through it. The condition is checked before each pass; watch **value** and **years** change:

```
value = 100
years = 0
while value < 150:
    value *= 1.10
    years += 1
```

VARIABLES

value = 110.00

years = 1

grow value, count a year

Watching a while loop

Step through it. The condition is checked before each pass; watch **value** and **years** change:

```
value = 100
years = 0
while value < 150:
    value *= 1.10
    years += 1
```

VARIABLES

value = 121.00

years = 2

still below 150, go again

Watching a while loop

Step through it. The condition is checked before each pass; watch **value** and **years** change:

```
value = 100
years = 0
while value < 150:
    value *= 1.10
    years += 1
```

VARIABLES

value = 133.10

years = 3

still below 150, go again

Watching a while loop

Step through it. The condition is checked before each pass; watch **value** and **years** change:

```
value = 100
years = 0
while value < 150:
    value *= 1.10
    years += 1
```

VARIABLES

value = 146.41

years = 4

still below 150, go again

Watching a while loop

Step through it. The condition is checked before each pass; watch **value** and **years** change:

```
value = 100
years = 0
while value < 150:
    value *= 1.10
    years += 1
```

VARIABLES

value = 161.05

years = 5

grow value, count a year

Watching a while loop

Step through it. The condition is checked before each pass; watch **value** and **years** change:

```
value = 100
years = 0
while value < 150:
    value *= 1.10
    years += 1
```

VARIABLES

value = 161.05

years = 5

161.05 < 150 ? no, the loop stops

Careful: it must be able to stop

Something inside the loop has to move the condition towards **False**, or it runs forever:

```
value = 100
while value < 150:
    years += 1      # value never changes, so this never stops
```

A **while** loop needs an exit. Make sure each pass changes something the condition depends on. If a cell seems stuck, interrupt it with the stop button.

Your turn 3

How many years to grow **1000** into **1500** at **8%** a year? Complete the while loop.

Python Code

```
value = 1000
years = 0
while value < 1500:
    ...
    years
```

Two lines in the body: grow `value`, then count the year.

Your turn 3: answer

```
value = 1000
years = 0
while value < 1500:
    value *= 1.08
    years += 1
years
```

Result: 6. After five years the value is still under 1500; the sixth year pushes it above.

4 • Your own functions

PART 4

Naming a calculation once, so you can reuse it anywhere.

You already use functions

`print`, `len`, `max`, and `sum` are functions: a name, then inputs in brackets. Today you write your **own**.

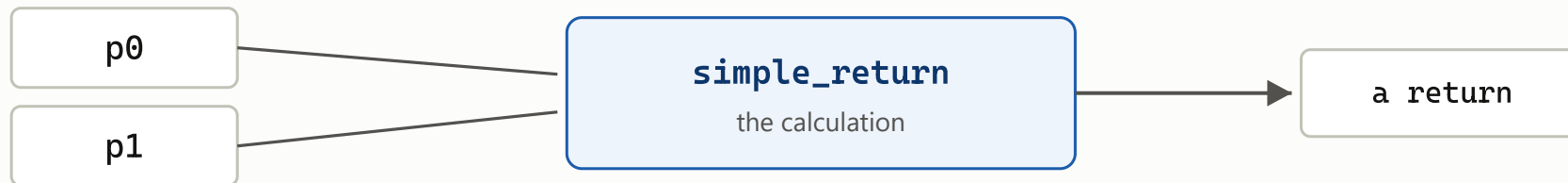
Defining a function

`def` creates a function. You give it a name, its inputs, and what it returns:

Python Code

```
def simple_return(p0, p1):  
    return (p1 - p0) / p0  
  
simple_return(183.56, 182.19)
```

-0.007463499673131426



The shape of a def

```
def simple_return(p0, p1):      # name, and inputs (the parameters)
    return (p1 - p0) / p0      # the value it hands back
```

- `def name(parameters):` opens the function
- the indented body is what it does
- `return` hands a value back to whoever called it

Call it as often as you like

Once defined, a function can be used again and again:

Python Code

```
def simple_return(p0, p1):  
    return (p1 - p0) / p0  
  
print(simple_return(100, 110))  
print(simple_return(50, 45))
```

```
0.1  
-0.1
```

Write the formula once, and trust it everywhere. There is no copy to get wrong.

Parameters and arguments

The names in the `def` are **parameters**; the values you pass in are **arguments**:

```
def simple_return(p0, p1):    # p0 and p1 are parameters
    return (p1 - p0) / p0

simple_return(100, 110)       # 100 and 110 are arguments
```

Inside that call, `p0` is 100 and `p1` is 110.

return hands a value back

Without **return**, a function does its work but gives nothing back:

Python Code

```
def show(p0, p1):  
    print((p1 - p0) / p0)    # prints, but does not return  
  
x = show(100, 110)  
print(x)
```

0.1

None

x is **None**: **show** printed a value but did not **return** one. Use **return** when you want to keep the result.

Too few arguments **PREDICT**

`simple_return` needs two inputs. What happens if you give it only one? Predict, then run.

Python Code

```
def simple_return(p0, p1):  
    return (p1 - p0) / p0  
  
simple_return(100)
```

Python will not guess the missing value.

Default arguments

A parameter can have a **default**, used when you do not pass it:

Python Code

```
def grow(price, rate=0.05):  
    return price * (1 + rate)  
  
print(grow(100))          # uses the default rate of 5%  
print(grow(100, 0.10))    # passes 10% instead
```

```
105.0  
110.000000000000000001
```

rate=0.05 in the **def** sets the default. Passing a value overrides it.

Documenting a function

A **docstring** is a short description in triple quotes, just under the **def**:

Python Code

```
def simple_return(p0, p1):  
    """Percentage change in price from p0 to p1."""  
    return (p1 - p0) / p0  
  
help(simple_return)
```

Help on function simple_return:

```
simple_return(p0, p1)  
    Percentage change in price from p0 to p1.
```

A function can contain a loop

Functions and loops combine. Here is the average of a list, as a function:

Python Code

```
def average(values):  
    total = 0  
    for v in values:  
        total += v  
    return total / len(values)  
  
average([10, 20, 30])
```

20.0

The name **average** now stands for four lines of work.

Your turn 4

Write a function `total_return(prices)` that returns the return from the first price to the last.

Python Code

```
def total_return(prices):  
    ...  
  
total_return([100, 104, 103, 110])
```

The first price is `prices[0]`, the last is `prices[-1]`.

Your turn 4: answer

```
def total_return(prices):  
    return (prices[-1] - prices[0]) / prices[0]  
  
total_return([100, 104, 103, 110])
```

Result: 0.10. The function works for a list of any length, because `prices[-1]` is always the last one.

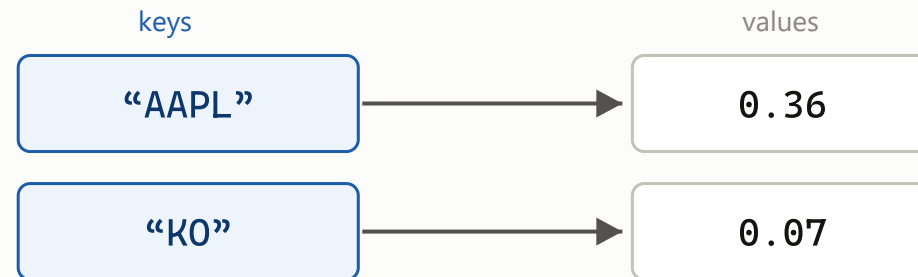
5 • Dictionaries

PART 5

Storing values by name, instead of by position.

Positions, or names

A list finds a value by its **position**, `prices[0]`. A **dictionary** finds a value by a **key** you choose, such as a stock's name.



Making a dictionary

Curly braces, holding **key: value** pairs:

Python Code

```
year_return = {"AAPL": 0.36, "KO": 0.07}  
year_return
```

```
{'AAPL': 0.36, 'KO': 0.07}
```

Here the keys are tickers, and the values are their yearly returns.

Looking up a value

Use the key in square brackets, the way a position works for a list:

Python Code

```
year_return = {"AAPL": 0.36, "KO": 0.07}  
year_return["AAPL"]
```

0.36

A key like **"AAPL"** is usually clearer than a bare number such as **[0]**.

Adding and updating

Assign to a new key to add a pair; assign to an existing key to change it:

Python Code

```
year_return = {"AAPL": 0.36, "KO": 0.07}  
year_return["NVDA"] = 1.80  
year_return
```

```
{'AAPL': 0.36, 'KO': 0.07, 'NVDA': 1.8}
```

Looping over a dictionary

A **for** loop over a dictionary walks its **keys**:

Python Code

```
year_return = {"AAPL": 0.36, "KO": 0.07}
for ticker in year_return:
    print(ticker, year_return[ticker])
```

```
AAPL 0.36
KO 0.07
```

ticker takes each key in turn, and **year_return[ticker]** reads its value.

A dictionary can hold lists

Values can be anything, including whole price series:

Python Code

```
prices = {  
    "AAPL": [183.56, 182.19, 179.87],  
    "KO": [55.64, 55.77, 55.58],  
}  
prices["KO"]
```

```
[55.64, 55.77, 55.58]
```

One object now holds every stock's prices, each under its own name.

Your turn 5

Add Coca-Cola's volatility of **0.011** to the dictionary, then read it back.

Python Code

```
vol = {"AAPL": 0.020}
```

```
...
```

```
vol["KO"]
```

Your turn 5: answer

```
vol = {"AAPL": 0.020}
```

```
vol["KO"] = 0.011
```

```
vol["KO"]
```

Result: 0.011. Assigning to a new key adds it; looking it up reads it back.

6 • Measuring risk properly

PART 6

Everything from today, combined into a real measure of risk.

From range to volatility

Last session's risk gauge used only two days, the highest and the lowest. A better measure uses **every** day's return. That measure is **volatility**: how much the daily returns typically spread around their average.

What volatility is

Volatility is the **standard deviation** of the daily returns:

$$\sigma = \sqrt{\frac{1}{n} \sum_i (r_i - \bar{r})^2}$$

In words: take each return's distance from the average, square it, average those squares, and take the square root. It is the typical size of a daily move.

The calculation, in three steps

Each step is something you can already do:

1. the **daily returns**, with a loop (Part 1)
2. their **average**, `sum(returns) / len(returns)`
3. each return's **squared distance** from the average, averaged, then square-rooted with `** 0.5`

We now build these into one function, one step at a time.

Build it up: the daily returns

Start the function with just step 1, the returns loop. It is not finished yet:

```
def volatility(prices):  
    returns = []  
    for i in range(1, len(prices)):  
        returns.append((prices[i] - prices[i - 1]) / prices[i - 1])  
    # still to come: the average, and the spread
```

So far it only collects the daily returns into a list.

Build it up: the average

Add step 2, the average return. Still not finished:

```
def volatility(prices):  
    returns = []  
    for i in range(1, len(prices)):  
        returns.append((prices[i] - prices[i - 1]) / prices[i - 1])  
    mean = sum(returns) / len(returns)  
    # still to come: the spread around the mean
```

Now it also has the average of those returns.

Build it up: the volatility

Add step 3, the spread, and **return** it. Now it is complete, so run it:

Python Code

```
def volatility(prices):  
    returns = []  
    for i in range(1, len(prices)):  
        returns.append((prices[i] - prices[i - 1]) / prices[i - 1])  
    mean = sum(returns) / len(returns)  
    squared = 0  
    for r in returns:  
        squared += (r - mean) ** 2  
    return (squared / len(returns)) ** 0.5  
  
volatility([183.56, 182.19, 179.87, 179.15, 183.48])
```

0.014300824856182556

Every idea from today is here: two loops, an empty list, **+=**, and a **return**.

Apple's volatility

Now reuse the function. Give it Apple's week of prices:

Python Code

```
aapl = [183.56, 182.19, 179.87, 179.15, 183.48]  
volatility(aapl)
```

```
0.014300824856182556
```

This calls the `volatility` function defined on the previous slide.

Coca-Cola's volatility

The same function, a different list:

Python Code

```
ko = [55.64, 55.77, 55.58, 55.50, 55.91]  
volatility(ko)
```

```
0.004115917006903075
```

Coca-Cola's real closes for the same five days. A much smaller number than Apple's: this price moved less from day to day.

Two stocks, one dictionary

Keep both answers together, each under its ticker:

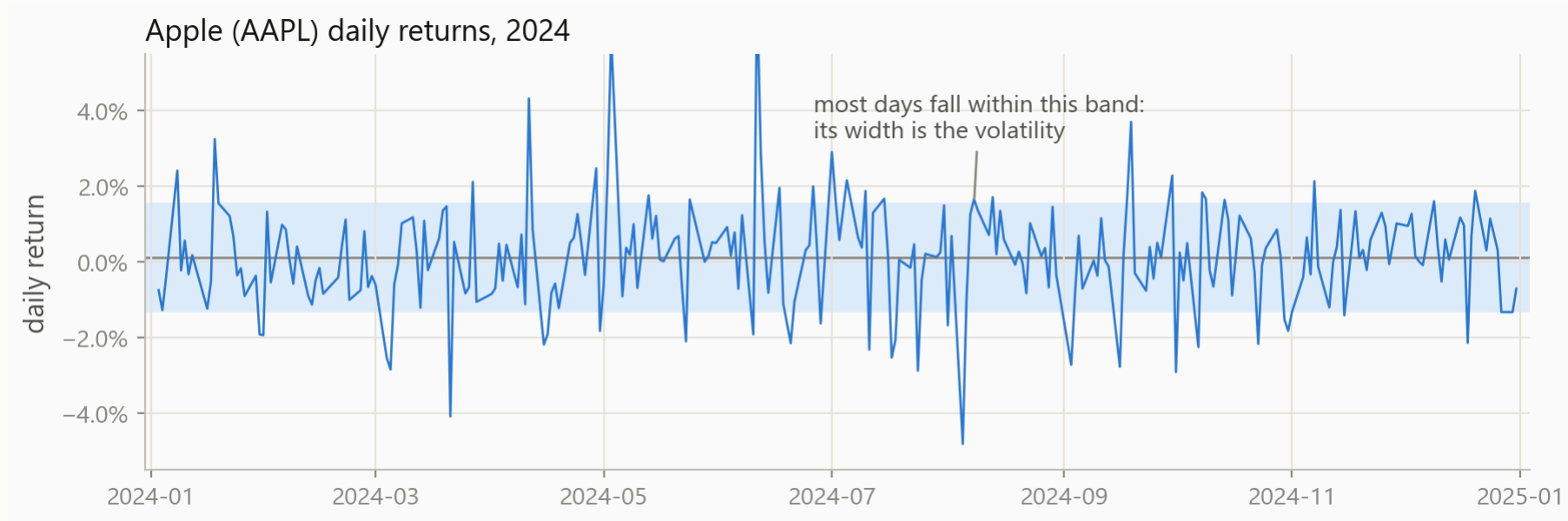
Python Code

```
vol = {"AAPL": volatility(aapl), "KO": volatility(ko)}  
vol
```

```
{'AAPL': 0.014300824856182556, 'KO': 0.004115917006903075}
```

Apple's number is the larger one. This is the volatility comparison the case asks you to build.

The returns that volatility measures



Each point is one day's return. Volatility is the width of the band they mostly fall inside.

What stays with you

For loops

`for x in items:` runs the body once per item.

if / elif / else

Run a block only when a condition holds.

Dictionaries

`{key: value}`, looked up by key, not position.

While loops

`while condition:` repeats until the condition is false.

Functions

`def name(args):` with `return`, plus defaults and a docstring.

Volatility

The standard deviation of the daily returns.

The course case, Part 2 **CASE**

The risk report continues. Last session you compared Apple and Coca-Cola with a rough price range. This session you replace it with a proper measure:

Build a volatility calculation from the daily returns, run it on each stock, and organise the results in a dictionary. Still no pandas: just today's loops and functions.

Case Part 2: this week

`session_02_case.ipynb`, done at home before Session 3:

- reload Part 1's data and the findings you saved
- build the daily-return loop into a function of your own
- compute proper volatility for Apple and Coca-Cola, and compare it to the crude range from Part 1
- rank a few more stocks by risk, organised in a dictionary

Hints and full solutions are folded into the notebook. Work first, then check.

Before the next session

HOMEWORK

Exercises

`session_02_exercises.ipynb`: loops, if, while, functions, and dictionaries. Hints and solutions folded in.

HOMEWORK

Case Part 2

`session_02_case.ipynb`: the volatility of Apple versus Coca-Cola.

IF STUCK

Get help

Ask a friend, ask an AI for a hint, or email me at jobo@econ.au.dk.
Stuck over 15 minutes? Note it and bring it next session.

Next session: packages. pandas, numpy, and matplotlib do all of this on eleven stocks at once, and plot the results.